

LAPORAN TUGAS – FINAL PROJECT

Perancangan Analisis Algoritma – B

Dosen : Rully Soelaiman, S.Kom., M.Kom.

Maulana Ikhsan – 5025241163

Daftar Isi.....	
1. Masalah.....	1
a. Problem description	1
b. Problem analysis	2
2. Penyelesaian	3
a. Mathematical approach.....	3
b. Algorithmic approach	6
c. I/O and variable handling.....	9
3. Coding.....	11
a. Pseudocode	11
b. Complete program	13
4. Hasil	23
a. Correctness analysis.....	18
b. Complexity analysis.....	19
c. Solution improvement analysis.....	20
d. Submission proof	21

1. Masalah

a. Problem description

Pada final project ini diberikan soal dari platform SPOJ yaitu :

- Nama : NKDIST – Distance
- Link : <https://www.spoj.com/problems/NKDIST/>
- Ss :

NKDIST - Distance

no tags

[English](#) [Vietnamese](#)

Consider a sequence D consisting of an infinite number of hexadecimal digits made by concatenating all the positive integers 1, 2, 3, 4,..., N,...

The sequence D begins with:

123456789ABCDEF101112131415161718191A1B1C1D1E1F202122...

We may see D as an infinite string of hexadecimal digits. Let S be an arbitrary string consisting only of hexadecimal digits. The number of occurrences of S in D as a substring is infinite. The distance between two nonoverlapping occurrence of S is the number of digits between these two occurrences. For instance, if S='21', the distance between the first two occurrences of S is 27 (as illustrated above).

Task

You are given a string S of at most 30 characters long. Write a program that determines the distance between the first two occurrences of S in D.

Input

The input contains the string S in a single line.

Output

The output contains the distance between the first two occurrences of S in D in a single line.

Example

Input	21
Output	27
Input	A
Output	26

[Submit solution!](#)

- Deskripsi soal :
Consider a sequence D consisting of an infinite number of hexadecimal digits made by concatenating all the positive integers 1, 2, 3, 4,..., N,...

The sequence D begins with:

123456789ABCDEF101112131415161718191A1B1C1D1E1F202122...

We may see D as an infinite string of hexadecimal digits. Let S be an arbitrary string consisting only of hexadecimal digits. The number of occurrences of S in D as a substring is infinite. The distance between two nonoverlapping occurrence of S is the number of digits between these two occurrences. For instance, if S='21', the distance between the first two occurrences of S is 27 (as illustrated above).

Task :

You are given a string S of at most 30 characters long. Write a program that determines the distance between the first two occurrences of S in D.

Input : The input contains the string S in a single line.

Output : The output contains the distance between the first two occurrences of S in D in a single line.

Example :

Input

21

Output

27

Input

A

Output

26

b. Problem analysis

Masalah diatas merupakan jenis problem divide and conquer untuk mencari jarak antara kemunculan pertama string S dan kemunculan kedua string tersebut pada baris hexadecimal

$D = 123456789ABCDEF1011121314151617181.....$

Misal input $S = "12"$, maka output dari masalah diatas adalah

$D = \text{12}3456789ABCDEF1011\text{12}1314151617181 \dots$

Dapat dilihat bahwa jarak antara kemunculan pertama dan kedua adalah selisih dari jarak antara index terakhir kemunculan pertama dan index pertama kemunculan kedua yaitu

$Dist = D_1[n - 1] - D_2[0]$ atau bisa disebut sebagai jarak antara kemunculan pertama dan kedua, lalu ditambah panjang dari kemunculan pertama, amka :

$$Dist = D_1[n - 1] - D_2[0] \leftrightarrow D_1[0] - D_2[0] - n \leftrightarrow first - second - n$$

Berdasarkan constraint yang ada, diketahui bahwa $0 \leq len(s) \leq 30$. Maka dengan melakukan analisis batas, dapat disimpulkan bahwa

- Lower bound : $S = "1" \rightarrow 14$ char jarak first dan second
- Upper bound : $S = "FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF"$ $\rightarrow$ estimasi kasar kemunculan pertama pada char ke 2×10^{37} dalam sequence D(penjelasan terkait hal ini akan dibahas pada bab 2.a).
- Maximum search : Dengan input "S" dengan panjang m, maka paling maksimal kemunculan kedua dari "S" akan berada pada panjang m+1(worst case)
- Worst case ex : Ex. "FFF" $\rightarrow$ "1FFF"

Dengan begini, diketahui terdapat 3 hal yang harus dilakukan yaitu :

- Hitung kemunculan pertama string S pada sequence (first)
- Hitung kemunculan kedua string S pada sequence (second)
- Menyimpan nilai dalam variabel dengan kapasitas $X, X \geq 10^{38}$

2. Penyelesaian

a. Mathematical approach

Untuk mencari posisi angka N pada sequence, dapat digunakan 2 approach:

1. Brute force

Untuk mencari ini, dapat dibuat fungsi untuk generate semua kemungkinan string mulai dari 1 $\rightarrow$ infinity dan melacak posisinya menggunakan variabel sementara. Tetapi hal ini tidaklah efektif karena produksi string dapat mencapai angka yang sangat besar. Maka dari itu untuk ini dan selanjutnya dapat disimpulkan menggunakan pendekatan matematis dengan metode divide and conquer

2. Mathematic generation

Untuk mencari ini dapat dilakukan dengan melakukan pengibaratan menggunakan desimal, diketahui bahwa

- Terdapat 9 angka satuan pada bilangan asli $X \rightarrow X_1 \in [0,9]$
- Terdapat 99 angka puluhan, atau bisa disebut sebagai $|X_2| = 99 \rightarrow |X_2| = 90 + X_1 \rightarrow |X_2| = 10 \times X_1 + X_1$
- Jika sequence diatas dijabarkan, maka akan menjadi seperti tabel

Labels	Digits	Total Numbers	Expression
X1	1	9	9×10^0
X2	2	99	$(9 \times 10^1) + (9 \times 10^0)$
X3	3	999	$(9 \times 10^2) + (9 \times 10^1) + (9 \times 10^0)$
...	...	...	
X(n)	n	...	$(9 \times 10^{n-1}) + (9 \times 10^{n-2}) + \dots + (9 \times 10^0)$

- Sehingga dapat dirumuskan, untuk sembarang bilangan dengan panjang k, terdapat $\sum_{n=1}^k 9 \times 10^{n-1}$ bilangan bulat
- Jika diperhatikan, untuk setiap k digit, terdapat karakter sebanyak total sebanyak $K \times$ expression pada derajat k tersebut. Sehingga dapat dihitung jumlah karakter yang ada untuk semua bilangan dengan panjang K yaitu sebanyak

$$\sum_{n=1}^k n \times 9 \times 10^{n-1} \rightarrow 9 \times \sum_{n=1}^k n \times 10^{n-1}$$

Karakter untuk bilangan dengan basis 10.

- Hal yang serupa juga dapat dilakukan untuk bilangan hexadecimal berbasis 16 dengan 15 buah digit satuan, yaitu dengan mengganti pengali nya saja.

Berdasarkan soal, diketahui posisi string “S” akan berada pada posisi setelah S-1. Misalkan S merupakan string yang mengandung K digit, maka posisi pasti kemunculan angka S pada string dapat dirumuskan sebagai jumlah semua karakter yang memiliki digit < digit(S)[N] ditambah dengan jumlah karakter yang kurang dari S dengan digit == digit(S)[X]. Maka $pos(S) = N + X$, maka

$$N = \sum_{x=1}^{k-1} X \times 15 \times 16^{X-1} \rightarrow$$

$$N = 15 \times \sum_{x=1}^{k-1} X \times 16^{X-1} \rightarrow 15 \times M$$

$$M = \sum_{x=1}^{k-1} X \times 16^{X-1} \rightarrow (1 \times 16^0) + (2 \times 16^1) + \dots + ((k-1) \times 16^{k-1})$$

$$16 \times M = 16 \times \sum_{x=1}^{k-1} X \times 16^{X-1} \rightarrow (1 \times 16^1) + (2 \times 16^2) + \dots + ((k-1) \times 16^k)$$

$$16 \times M = \sum_{x=1}^{k-1} X \times 16^X \rightarrow (1 \times 16^1) + (2 \times 16^2) + \dots + ((k-1) \times 16^k)$$

$$16M - M = [(1 \times 16^1) + (2 \times 16^2) + \dots + ((k-1) \times 16^k)] - [(1 \times 16^0) + (2 \times 16^1) + \dots + ((k-1) \times 16^{k-1})]$$

$$16M - M = -1 \times 16^0 - 1 \times 16^1 - \dots - 1 \times 16^{k-2} + (k-1) \times 16^{k-1}$$

Ibaratkan G sebagai deret geometri pada 15M berisi $G = -1 \times 16^0 - 1 \times 16^1 - \dots - 1 \times 16^{k-2}$

Maka, dengan

$A = -1$; $R = 16$. Nilai dari 15M dapat dihitung sebagai jumlahan dari G dan juga $(k-1) \times 16^{k-1}$. Maka :

$$G = -1 \times 16^0 - 1 \times 16^1 - \dots - 1 \times 16^{k-1}$$

$$S_G = \frac{-1 \times (16^{k-1} - 1)}{(16 - 1)} = \frac{-16^{k-1} - 1}{15}$$

Maka,

$$15M = \frac{-16^{k-1} - 1}{15} + (k-1) \times 16^{k-1}$$

Berdasarkan perhitungan diatas, didapatkan

$$N = 15M = \frac{-16^{k-1} - 1}{15} + (k-1) \times 16^{k-1}$$

Maka dengan ini sudah berhasil mencari untuk N. Lalu untuk X, diketahui banyaknya angka yang muncul sebelum string “S” dengan k digit adalah sebanyak $n - 16^{k-1}$. Maka banyaknya karakter yg muncul sebelum string “S” dengan K digit adalah

$$X = k \times (n - 16^{k-1})$$

Maka, posisi kemunculan angka “S” pada sequence terdapat pada index I dimana dalam bentuk closed form nya menjadi

$$I = \left(\frac{-16^{k-1} - 1}{15} + (k-1) \times 16^{k-1} \right) + (k \times (n - 16^{k-1}))$$

Atau dalam bentuk open form menjadi

$$I = \left(\sum_{x=1}^{k-1} X \times 15 \times 16^{x-1} \right) + (k \times (n - 16^{k-1}))$$

Berdasarkan analisis masalah pada 1.b, setelah mencari posisi absolut “S” maka akan dicari kemunculan pertama dari string S pada sequence. Terdapat 3 kemungkinan yang membuat string S muncul pada infinite sequence D, misal pada sequence S = “213” yaitu :

1. Muncul sebagai angka utuh
Ex. “...211212**213**21421...”

Kemunculan string “S” pada angka utuh terbagi menjadi beberapa kemungkinan yaitu :

- Nilai(S) secara utuh → Ex. “213”
Untuk ini maka value $n = \text{num}(S)$ tanpa offset(pergeseran bilangan apapun diawal index).
- Nilai(S) + random tail → Ex. “213A”
Untuk ini maka value $n = 16 \times \text{num}(S) + t, t \in [0, F]$ tanpa offset
- Random head + Nilai(S) → Ex. “A213”.
Untuk ini maka value $n = t \times 16^m + \text{num}(S), t \in [1, F]$ dengan 1 offset didepan.

2. Potongan parsial S pada n + potongan utuh pada n+1
Ex. “...10111**213**141...”

Kemunculan string “S” pada kasus ini dapat dianalisa dengan membagi S menjadi tail dan rest, misal input “213”. Maka tail = “1**2**” dan rest = “**13**”. Maka untuk sembarang angka tail :

- $\text{Len}(S) = m$
- $\text{Len}(\text{tail}) = a$
- $\text{Len}(\text{rest}) = m - a$ (len input dikurangi yg sudah terpakai di tail)

Misalkan R merupakan sebuah bilangan utuh(bukan merupakan potongan substring angka lain dari $y = \text{tail} + 1$ dengan panjang L, maka jumlah kemungkinan himpunan $H(y)$ adalah

$H(y) = R[(0 \dots l) - 1]$ dengan offset didepan sepanjang $\text{len}(y) - a$ karakter → (maksudnya adalah offset akan terjadi mulai dari digit pertama pada angka y hingga digit ke-x pada angka y yang menjadi kemunculan pertama pada tail).

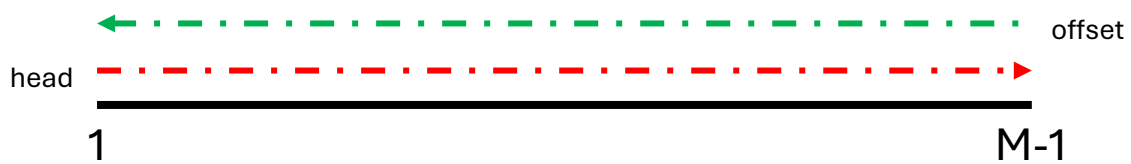
Maka kandidat hasil akan berada di $n = y - 1$ dengan $\text{offset} = \text{len}(n) - a$.

3. Potongan parsial S pada angka n dan n+1
Ex. “...3203**213**2232332...”

Kemunculan string “S” pada kasus ini dapat dianalisa dengan membagi S menjadi tail dan head (mirip seperti kasus2, tetapi tidak full memotong), misal input “213”. Maka tail = “321” dan head = “322”. Maka untuk tail :

- $\text{Len}(S) = m$
- $\text{Len}(\text{tail}) = a$
- $\text{Len}(\text{head}) = m - a$

Misalkan head merupakan prefix dari $y = n+1$ (sama seperti kasus2) dengan panjang L, maka y akan berada dalam range $S[1] \rightarrow S[m-1]$ (ini iterasi untuk mencari kemungkinan pembagian tail dan head) dengan offset sejauh $[m-1, 1]$.



Maka kandidat valid untuk kasus ini adalah y yang memenuhi syarat :

1. Syarat interval Y

Interval y berada pada

$$y \in [\min, \max] \rightarrow y \in [\text{left} \times 16^{l-\text{len}(\text{head})}, (\text{left} + 1) \times 16^{l-\text{len}(\text{head})} - 1]$$

note *misal min = “EA****”, maka max = “(EB0000)-1” = “EAFFFF”

2. Syarat kongruensi nilai

Untuk setiap looping y pada interval yang ada akan dilakukan pengecekan nilai berdasarkan sifat kongruensi bilangan

- Karena head adalah prefix $y=n+1$, maka tail adalah suffix dari $y=n$
- Karena tail merupakan a digit terakhir dari n (lihat pada Ex., $a=2$) maka nilai desimal dari digit terakhir n adalah tail $\rightarrow n \bmod 16 \equiv \text{tail}$
- Maka, $n \bmod 16 \equiv \text{tail} \rightarrow y = n + 1 \rightarrow y \equiv (\text{tail} + 1) \bmod 16^a$

Jika kandidat angka y pada kasus ini memenuhi 2 syarat diatas, maka dapat dihitung semua kemungkinan kandidat valid y dengan (offset = $\text{len}(n) - a$) karakter dari depan.

Dengan melakukan hal diatas, maka telah berhasil dicari semua kemungkinan kemunculan S dan perhitungan posisi S dengan akurat dan efisien menggunakan approach Divide and Conquer.

b. Algorithmic approach

Algorithmic approach yang digunakan untuk problem ini adalah pendekatan dengan metode Divide and Conquer serta penyelesaian menggunakan approach Branch and

Bound untuk semi-simulasi pruning pada kandidat jawaban yang ada dengan metode candidate enumeration seperti dibawah.

1. Overview algoritma

Algoritma untuk penyelesaian masalah ini bekerja secara 4 fase yaitu :

1. Precompute

Di fase ini dilakukan perhitungan untuk 16^x untuk mencegah miscalculation pada fungsi di algoritma serta menghemat waktu komputasi

2. String generation(candidate)

Di fase ini dilakukan analisis kasus kemungkinan valid string S pada sequence D dan perhitungan perkiraan substring valid berdasarkan perhitungan matematis pada 2.a

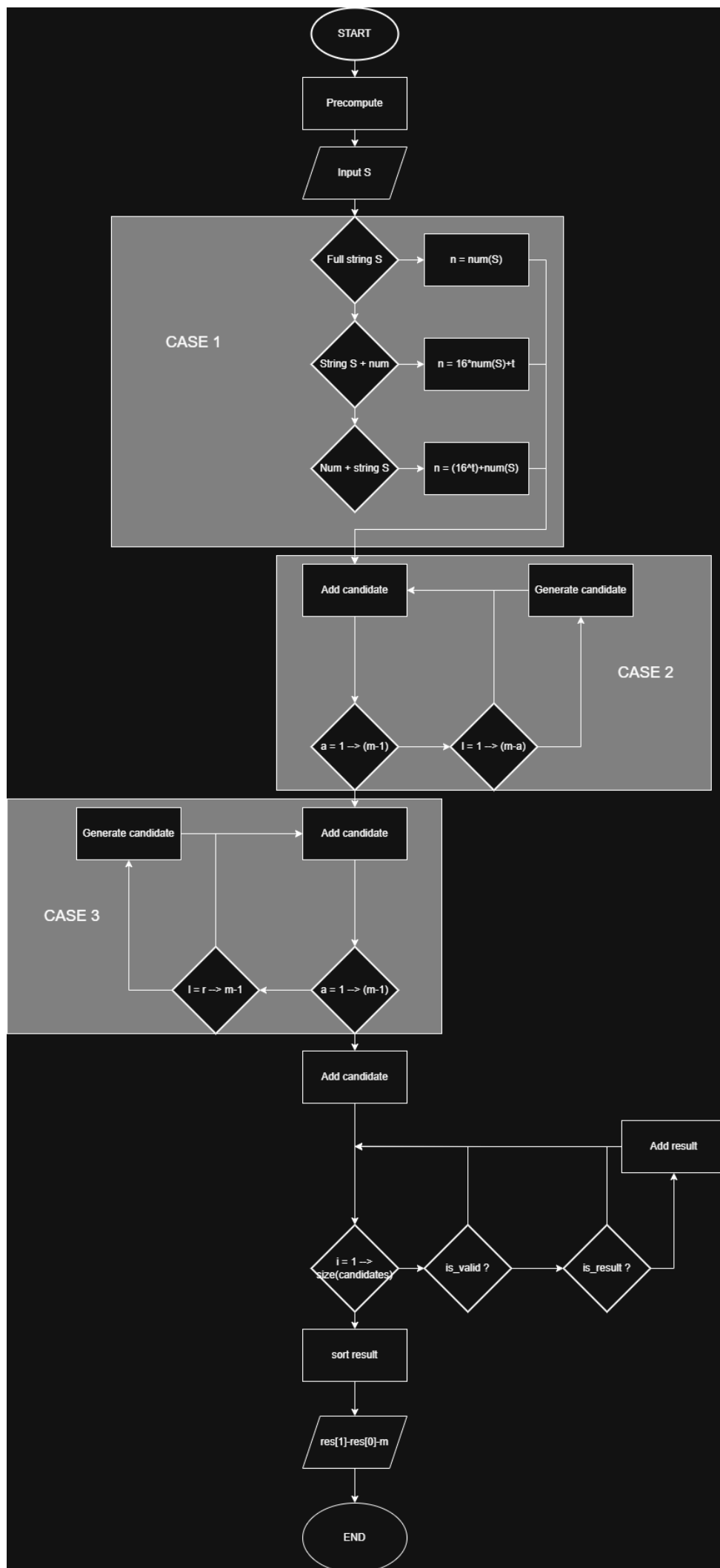
3. Seleksi jawaban

Di fase ini dilakukan perhitungan posisi pasti dari kemungkinan jawaban pada fase 2 dengan menggunakan offset sebagai acuan pergeseran substring valid dari kemungkinan penyusunan string pada sequence D

4. Kalkulasi jawaban untuk output

Di fase ini hanya menerapkan rumus jarak yang ada pada 1.b

Berikut merupakan flowchart lengkap dari algoritma untuk penyelesaian masalah pada bab1.



2. Penerapan prosedur competitive programming

Pada program untuk penyelesaian problem ini, penulis menggunakan approach procedural programming untuk menargetkan time complexity serta memory complexity yang efektif dan efisien. Berikut merupakan detail penerapan konsep competitive programming pada kode penulis.

Metode pendekatan programming :

- Program dibagi menjadi fungsi independen dimana setiap fungsi memiliki tugas spesifik
- Modularitas program diatur berbasis fungsi pada setiap program
- Control flow diatur eksplisit di “main()”
- Efisiensi memori dilakukan dengan tidak menggunakan overhead seperti pada approach OOP

Penjelasan efektivitas programming

- Eliminasi bruteforce
Dilakukan eliminasi metode bruteforce untuk generasi hasil dengan hanya membangun window kecil dari substring calon kandidat untuk verifikasi hasil.
- Strategi enumerasi kandidat
Untuk menghindari pencarian substring secara langsung dilakukan perhitungan matematis pada case1-case3 untuk pencarian posisi absolut dari calon jawaban berdasarkan persamaan matematis pada bab 2.a sehingga dipastikan kandidat yang ada merupakan solusi “false positive” dan juga “true positive”.
- Penggunaan BigInt
Digunakan BigInt yang direpresentasikan menggunakan library int128 dari C++ untuk handling nilai besar sesuai dengan constraint jawaban maksimal yang ada yaitu
 $\text{Max digit} = \log_2(15^{30}) \approx 117 \text{ digit biner}$. Ini memenuhi limit dari int128 yaitu 128 digit biner.
- Optimasi library
Operasi dilakukan menggunakan fungsi primitive dasar yang berada dalam lingkup bahasa C sehingga dapat diproses secara singkat menggunakan library bawaan pada C++.

c. I/O and variable handling

- Input
Input diambil dengan `char temp[32]` agar bisa dilakukan menggunakan `scanf("%s", temp);` lalu di parse menuju string S untuk perhitungan hasil operasi. Sebelumnya sudah sempat dicoba `scanf` langsung ke S tetapi

menimbulkan error, amka dari itu hal ini dilakukan agar input nya menjadi aman.

- Output

Output di-print dalam bentuk string hasil yang merupakan angka numerik sesuai dengan bab 1.b dengan perhitungan dalam bentuk class BigInt agar bisa diimplementasikan dengan mudah.

- Variable

Variabel yang ada dan berkaitan dengan program diurus menggunakan unsigned int128 pada C++ untuk menghemat memori yang ada. Tetapi dikarenakan terdapat kemungkinan overflow untuk kasus besar maka hasil akhir posisi index kandidat ke-i disimpan dalam class BigInt yang ada.

Digunakan juga komparator dengan struct mindec agar secara default kandidat jawaban yang ada pada class Algo disimpan secara sorted.

3. Coding

a. Pseudocode

Beirkut merupakan pseudocode berisi rancangan masing masing fungsi serta algoritma akhir dari program yang penulis buat.

- Case 1 candidate calculation

```
PROCEDURE Case1

  IF first character of input ≠ '0' THEN
    ADD candidate (inputnum, offset 0)

    prefix ← inputnum × 16
    FOR t ← 0 TO 15 DO
      ADD candidate (prefix + t, offset 0)
    END FOR
  END IF

  FOR h ← 1 TO 15 DO
    n ← h × 16m + inputnum
    ADD candidate (n, offset 1)
  END FOR

END PROCEDURE
```

- Case 2 candidate calculation

```
PROCEDURE Case2

  FOR a ← 1 TO m-1 DO
    IF input[a] = '0' THEN CONTINUE

    y ← 0
    FOR l ← 1 TO m-a DO
      y ← y × 16 + value of next digit
      ADD candidate (y-1, offset = hexlen(y-1) - a)
    END FOR
  END FOR

END PROCEDURE
```

- Case 3 candidate calculation

```
PROCEDURE Case3

  FOR a ← 1 TO m-1 DO
    IF input[a] = '0' THEN CONTINUE
```

```

        COMPUTE mask, left, right based on prefix

        FOR l ← r+1 TO m+2 DO
            COMPUTE interval [low, high]

            FIND first valid y in interval using modular
alignment

            WHILE y ≤ high DO
                n ← y - 1
                ADD candidate (n, offset = hexlen(n) - a)

                IF cannot jump further THEN BREAK
                y ← y + step
            END WHILE
        END FOR
    END FOR

END PROCEDURE

```

- Final pseudocode

```

BEGIN PROGRAM

    CALL Precompute()

    READ input string S
    m ← length(S)

    // Convert input to numeric representation
    pref[0] ← 0
    value ← 0
    FOR i ← 0 TO m-1 DO
        digit ← hexadecimal value of S[i]
        hval[i] ← digit
        value ← value × 16 + digit
        pref[i+1] ← value
    END FOR
    inputnum ← value

    // Generate all possible candidate positions
    CALL Case1()
    CALL Case2()
    CALL Case3()

    // Sort all candidate positions
    SORT pool using numeric string comparison

```

```

// Get first occurrence
first ← pool[0]

// Compute minimum valid second occurrence
limit ← first + m

FOR each element in pool starting from index 1 DO
    IF element ≥ limit THEN
        second ← element
        BREAK
END FOR

// Compute final distance
result ← second - first - m
PRINT result

END PROGRAM

```

b. Complete program

Berikut merupakan full code penulis terhadap problem ini

```

#include <stdio>
#include <string>
typedef unsigned __int128 u128;

static u128 inputnum, pref[32], exp16[32];
static char input[32], prepos[32][48], pool[4096][64];
static int m, poolcnt, preposlen[32];
static unsigned char hval[32], poollen[4096];
static const char HEXD[] = "0123456789ABCDEF";

int hexlen(u128 x) {
    int len = 0;
    do { ++len; x >>= 4; } while (x);
    return len;
}

int u128tostr(char *dst, u128 x) {
    if (!x) { dst[0]='0'; dst[1]=0; return 1; }
    char buf[42]; int k=0;
    while (x) { buf[k++]=(char)('0'+(int)(x%10)); x/=10; }
    for (int i=0; i<k; ++i) dst[i]=buf[k-1-i];
    dst[k]=0;
    return k;
}

int dadd(char *res, const char *a, int la, const char *b, int lb)
{

```

```

    int i=la-1, j=lb-1, carry=0, k=0;
    char buf[48];
    while (i>=0 || j>=0 || carry) {
        int s = carry;
        if (i>=0) s += a[i--]-'0';
        if (j>=0) s += b[j--]-'0';
        buf[k++] = (char)('0'+(s%10));
        carry = s/10;
    }
    for (int i2=0; i2<k; ++i2) res[i2]=buf[k-1-i2];
    res[k]=0;
    return k;
}

int dsub(char *res, const char *a, int la, const char *b, int lb)
{
    int i=la-1, j=lb-1, borrow=0, k=0;
    char buf[48];
    while (i>=0) {
        int d = a[i--]-'0'-borrow;
        if (j>=0) d -= b[j--]-'0';
        if (d<0) { d+=10; borrow=1; } else borrow=0;
        buf[k++] = (char)('0'+d);
    }
    while (k>1 && buf[k-1]=='0') --k;
    for (int i2=0; i2<k; ++i2) res[i2]=buf[k-1-i2];
    res[k]=0;
    return k;
}

int dcmp(const char *a, int la, const char *b, int lb) {
    if (la!=lb) return la<lb ? -1 : 1;
    for (int i=0; i<la; ++i) { if (a[i]!=b[i]) return a[i]<b[i]
? -1 : 1; }
    return 0;
}

void precompute() {
    exp16[0]=1;
    for (int i=1; i<32; ++i) exp16[i]=exp16[i-1]<<4;

    prepos[0][0]='0'; prepos[0][1]=0; preposlen[0]=1;
    prepos[1][0]='0'; prepos[1][1]=0; preposlen[1]=1;
    for (int k=2; k<=31; ++k) {
        char term[42];
        u128 t = (u128)(k-1) * (u128)15 * exp16[k-2];
        int t1 = u128tostr(term, t);
        preposlen[k] = dadd(prepos[k], prepos[k-1], preposlen[k-
1], term, t1);
    }
}

```

```

    }
}

int calcpos(char *res, u128 n) {
    int k = hexlen(n);
    u128 tail = (u128)k * (n - exp16[k-1]);
    char ts[42]; int tl = u128tostr(ts, tail);
    return dadd(res, prepos[k], preposlen[k], ts, tl);
}

int verify(u128 n, int off) {
    int need = off + m;
    char window[128]; int wlen=0;
    u128 cur = n;
    while (wlen < need) {
        u128 x = cur;
        char buf[64]; int bk=0;
        do { buf[bk++] = HEXD[(int)(x&15)]; x>>=4; } while(x);
        for (int i=bk-1; i>=0; --i) window[wlen++] = buf[i];
        ++cur;
    }
    return memcmp(window+off, input, m)==0;
}

void addpool(const char *s, int sl) {
    for (int i=0; i<poolcnt; ++i) { if (poollen[i]==sl &&
memcmp(pool[i],s,sl)==0) return; }
    memcpy(pool[poolcnt], s, sl);
    pool[poolcnt][sl]=0;
    poollen[poolcnt]=(unsigned char)sl;
    ++poolcnt;
}

void add_res(u128 n, int off) {
    if (!n) return;
    int hl = hexlen(n);
    if (hl > m+1) return;
    if (off<0 || off>=hl) return;
    if (!verify(n, off)) return;
    char pos[48]; int pl = calcpos(pos, n);
    if (off) {
        char offstr[8]; int ol = u128tostr(offstr, (u128)off);
        char fp[48]; int fl = dadd(fp, pos, pl, offstr, ol);
        addpool(fp, fl);
    } else addpool(pos, pl);
}

void case1() {
    if (input[0]!='0') {

```



```

        add_res(inputnum, 0);
        u128 prefix = inputnum<<4;
        for (int t=0; t<16; ++t) add_res(prefix+(u128)t, 0);
    }
    for (int h=1; h<=15; ++h) add_res((u128)h*exp16[m]+inputnum,
1);
}

void case2() {
    for (int a=1; a<m; ++a) {
        if (input[a]=='0') continue;
        u128 y = 0;
        for (int l=1; l<=(m-a); ++l) {
            y = (y<<4) + (u128)hval[a+l-1];
            add_res(y-1, hexlen(y-1)-a);
        }
    }
}

void case3() {
    for (int a=1; a<m; ++a) {
        if (input[a]=='0') continue;
        int r = m-a;
        u128 mask = exp16[a]-1, tv = pref[a];
        u128 left = pref[m] - pref[a]*exp16[m-a];
        u128 right = (tv+1)&mask;

        for (int l=r+1; l<=m+2; ++l) {
            if ((l-r)<1 || (l-r)>=32) continue;
            u128 block = exp16[(l-r)];
            u128 low = left*block, high = (left+1)*block-1;
            if (high<low) continue;
            u128 rem = low&mask;
            u128 delta = (right+exp16[a]-rem)&mask;
            u128 y = low+delta;
            while (y<=high) {
                u128 n=y-1;
                add_res(n, hexlen(n)-a);
                if (high-y<exp16[a]) break;
                y += exp16[a];
            }
        }
    }
}

int main() {
    precompute();
    scanf("%s", input);
    m = (int)strlen(input);
}

```

```

    pref[0]=0;
    u128 iv=0;
    for (int i=0; i<m; ++i) {
        int v = (input[i] <= '9') ? (input[i]-'0') : (input[i]-
'A'+10);
        hval[i] = (unsigned char)v;
        iv = (iv<<4) + (u128)v;
        pref[i+1] = iv;
    }
    inputnum = iv;

    case1();
    case2();
    case3();

    for (int i=1; i<poolcnt; ++i) {
        int li = poollen[i];
        char key[64]; memcpy(key, pool[i], li+1);
        int j=i-1;
        while (j>=0 && dcmp(pool[j],poollen[j],key,li)>0) {
            int lj = poollen[j];
            memcpy(pool[j+1], pool[j], lj+1);
            poollen[j+1] = (unsigned char)lj;
            --j;
        }
        memcpy(pool[j+1], key, li+1);
        poollen[j+1] = (unsigned char)li;
    }

    char *first = pool[0]; int fl = poollen[0];
    char *second = 0;      int sl=0;
    char mstr[8];          int ml = u128tostr(mstr, (u128)m);
    char lim[48];          int ll = dadd(lim, first, fl, mstr,
ml);

    for (int i=1; i<poolcnt; ++i) {
        int il = poollen[i];
        if (dcmp(pool[i],il,lim,ll)>=0) {
            second = pool[i]; sl=il;
            break;
        }
    }

    char tmp[48]; int tl = dsub(tmp, second, sl, first, fl);
    char result[48]; dsub(result, tmp, tl, mstr, ml);
    printf("%s\n", result);
    return 0;
}

```

4. Hasil

a. Correctness analysis

Untuk membuktikan kebenaran program penulis, dilakukan 5 metode yaitu

1. Penetapan aturan dasar

- D merupakan infinite string yang dibentuk dari penggabungan bilangan natural hexadecimal berurutan tanpa leading 0.
 $D = 123456789ABCD\dots$
- S merupakan sembarang string hexadecimal dengan panjang m karakter dimana $m \leq 30$
- Kemunculan S dicatat dengan index awal kemunculan S pada D
- Jika kemunculan pertama S adalah P_1 dan kemunculan kedua S adalah P_2 , maka jawaban atas pertanyaan yang dibutuhkan adalah $P_2 - P_1 - m$

2. Pernyataan proposisi

“Untuk setiap input string S, algoritma akan mengembalikan jarak tepat diantara 2 kemunculan S pertama yang tidak bersinggungan”.

Untuk membuktikan ini, terdapat 3 hal untuk dibuktikan yaitu :

- Setiap kemungkinan kandidat yang dihasilkan algoritma merupakan kemunculan valid dari S
- Setiap kemunculan S dihasilkan dari salah satu kasus pada algoritma
- Program menghitung kemunculan valid pertama dan kedua lalu menghitung jaraknya secara akurat

3. Pemilihan strategi pembuktian

Digunakan “proof by exhaustion” untuk membuktikan kemunculan kandidat jawaban dan “direct proof” untuk membuktikan rumus evaluasi jarak. Dengan 3 lema(teorema pembantu) yang harus dibuktikan yaitu :

- Lema A; jumlah digit sebelum kemunculan S
- Lema B; setiap kemunculan kandidat jawaban harus memenuhi kasus pada algoritma
- Lema C; rumus jarak antara 2 kemunculan pertama

4. Eksekusi logika

- Lema A

Untuk sembarang angka N hexadecimal dengan panjang k, jumlah digit sebelum N terbagi mejadi 2 panjnag dengan rumus

1. $Len < k$

$$\sum_{d=1}^{k-1} d \times 15 \times 16^{d-1}$$

2. $Len == k$

$$k \times (n - 16^{k-1})$$

maka terdapat x buah digit dimana

$$X = \sum_{d=1}^{k-1} d \times 15 \times 16^{d-1} + k \times (n - 16^{k-1})$$

Terbukti

○ Lema B

Sebuah string S pada infinite hexadecimal string D, pasti muncul dengan salah diantara 2 kasus yaitu

- Muncul sebagai angka utuh
- Muncul sebagai angka parsial

Dengan semua variannya, terdapat total 4 kemungkinan yaitu

1. "...[S]..."
2. "...[S+'c']..."
3. "...['c'+S]..."
4. "...[...+T][H+...]...." → semua kemungkinan variasinya juga termasuk

Terbukti

○ Lema C

Misal terdapat sebuah string S dengan panjang m yang muncul pada index X_1 dan X_2 di D dan diasumsikan X_1 dan X_2 tidak bersinggungan. Maka

- Index awal X_1 = posisi X_1
- Index akhir X_1 = posisi $X_1 + m$
- Index awal X_2 = posisi X_2
- Index akhir X_2 = posisi $X_2 + m$

Selisih = index awal x_2 – index akhir x_1

Selisih = $X_2 - (X_1 + m) \rightarrow X_2 - X_1 - m$

Terbukti

5. Verifikasi

Dikarenakan semua poin pada no.2 sudah terbukti, makadapat dipastikan bahwa algoritma pada soal ini terbukti kebenarannya.

b. Complexity analysis

Berikut merupakan analisa kompleksitas dari program penulis untuk permasalahan NKDIST ini

○ Time complexity

Berikut merupakan time complexity untuk tiap komponen dengan operasi didalamnya. Dikarenakan limitasi pada input membuat $m \leq 30$, maka untuk worst case terdapat $30^3 = 27.000$ operasi yang dilakukan, tetap tergolong cepat dan efisien.

function	complexity	detail
memo()	$O(1)$	memoization total 32x loop
case1()	$O(m)$ $O(m^3)$	iterasi string murni valid

case2()		build up kandidat dari kombinasi potongan string
case3()	$O(m^3)$	build up kandidat dari kombinasi potongan string
result	$O(m^2)$	search kemungkinan kombinasi jawaban

- Memory complexity

Memory complexity untuk program ini akan berjumlah sesuai maksimum set possible yang digenerate oleh tiap kandidat. Untuk setiap set yang menyimpan $O(m^2)$ string BigInt(didapat dari case generate). Maka kompleksitas memori untuk program ini adalah sebesar $O(m^2)$.

c. Solution improvement analysis

Untuk mencapai time and memory complexity requirement, dilakukan beberapa perubahan serta perkembangan pada metode implementasi solusi program. Perkembangan yang dilakukan tidak sebagian maupun keseluruhan algoritma proses penyelesaian soal, melainkan hanya merubah metode implementasi menjadi approach yang lebih dasar serta dapat mudah dipahami.

Berikut merupakan perbandingan perkembangan terhadap program

Proses	Method		Improvement
	OOP	Procedural	
Program flow	Class abstraction dengan object based organization	Direct function calling	Menghilangkan overhead pada object management
Hexadecimal operator	String based	Char based dengan array of char	Penggunaan tipe data primitif mempercepat operasi dan manajemen memori terhadap sistem
Hexadecimal operation	String based dengan method pada class	Binary based dengan bitshift untuk utilitas basis-16(2^4 based)	Mempercepat operasi dengan mengubahnya menjadi binary-based
Time complexity	0.07s	0.05s	28.5% improvement

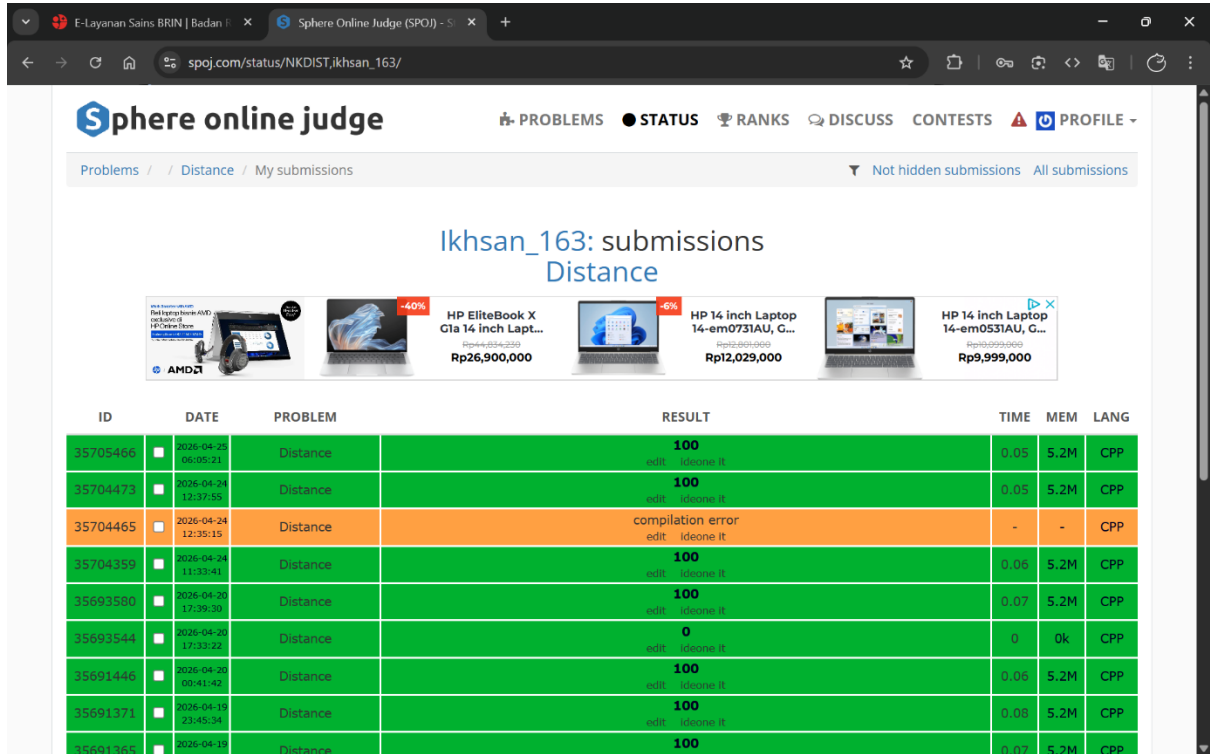
Secara keseluruhan, algoritma tetap berjalan dengan

- Time complexity : $O(m^3)$
- Memory complexity : $O(m^2)$

Tetapi dengan menerapkan operasi melalui cara dasar pada tipe data primitif dapat menghemat time overhead untuk pemanggilan fungsi pada tipe data tingkat tinggi sehingga dapat mempercepat waktu komputasi.

d. Submission proof

Berikut merupakan bukti submission dari problem ini



Sphere online judge | PROBLEMS | STATUS | RANKS | DISCUSS | CONTESTS | PROFILE

Problems / / Distance / My submissions | Not hidden submissions | All submissions

Ikhsan_163: submissions Distance






ID	DATE	PROBLEM	RESULT	TIME	MEM	LANG
35705466	2026-04-25 06:05:21	Distance	100 edit ideone.it	0.05	5.2M	CPP
35704473	2026-04-24 12:37:55	Distance	100 edit ideone.it	0.05	5.2M	CPP
35704465	2026-04-24 12:35:15	Distance	compilation error edit ideone.it	-	-	CPP
35704359	2026-04-24 11:33:41	Distance	100 edit ideone.it	0.06	5.2M	CPP
35693580	2026-04-20 17:39:30	Distance	100 edit ideone.it	0.07	5.2M	CPP
35693544	2026-04-20 17:33:22	Distance	0 edit ideone.it	0	0k	CPP
35691446	2026-04-20 00:41:42	Distance	100 edit ideone.it	0.06	5.2M	CPP
35691371	2026-04-19 23:45:34	Distance	100 edit ideone.it	0.08	5.2M	CPP
35691365	2026-04-19	Distance	100	0.07	5.2M	CPP